SYSTEMS DESIGN · REINFORCEMENT LEARNING · COMPUTER VISION · NLP · SOFT COMPUTING

NexusSim

Explained, Engineered — a real-city, real-time multi-agent traffic simulator, and its integrated extension into an AI-driven policy sandbox.

ENGINE

C++17 · Quadtree · SoA

PIPELINE

Python · OSM + real OD demand

EXTENSION

RL · CV · NLP · GA

OBJECTIVE

Speed + Equity, not just speed

This edition is generated from *docs/NexusSim_Explained.md*, the revised source document, and styled to match the established template. It adds the Requirements section, the 12-algorithm RL inventory (FR-16, Phases 19–24), and the updated implementation timeline. The template's layout and palette are unchanged.

CONTENTS

Table of Contents

How to read this document. Requirements carry traceable IDs (FR, NFR, TR, BR), and claims about the codebase are marked **[VERIFIED]** with a file:line citation, **[PLANNED]** for designed-but-unbuilt work, or **[NEW]** for content with no equivalent in the original PDF. Section 11 lists the assumptions behind this revision and what would resolve each one.

Revision notice. This is a rewrite of `docs/NexusSim_Explained.pdf` (generated earlier in this project's planning process), produced by reviewing that PDF's content against the actual codebase at `E:\Coding\Nexus_Sim` and against `IMPLEMENTATION_PLAN.md` Phases 0–18. Content is marked inline as follows: - **[VERIFIED]** — checked directly against source code in this repo; a file:line citation is given. - **[PLANNED]** — describes work in `IMPLEMENTATION_PLAN.md` Phases 12–18 that does not exist in the codebase yet. The original PDF did not consistently distinguish "built today" from "designed for later," which this revision corrects throughout. - **[NEW]** — content with no equivalent in the original PDF (the Requirements section, the phase-dependency timeline table, and all code citations are new). - **[ASSUMPTION]** — a judgment call made where the source material was silent or ambiguous; see §10 for the full list and what would resolve each one. No PDF or implementation-plan file was attached as a binary upload to this conversation; the baseline used is the `NexusSim_Explained.pdf` this same project thread generated, and the `IMPLEMENTATION_PLAN.md` / `docs/IMPLEMENTATION_PLAN.md` files present in the repository. If a *different* PDF was intended, see §10.

PART 1

What NexusSim Is

[VERIFIED, expanded with analogies — NEW explanatory content]

Think of NexusSim as a **flight simulator for city traffic**. Just as an airline doesn't let a new pilot learn maneuvers on a real passenger jet, a city shouldn't have to close a real street or repaint real bus lanes just to find out whether the idea works. NexusSim gives urban planners and researchers a place to "fly" a policy change — a new bus lane, a congestion charge, a smarter traffic light — inside a realistic, physics-driven copy of the city first.

Two things make it more than a toy model:

1. **The vehicles behave like real drivers, not dots on rails.** Each simulated vehicle follows the **Intelligent Driver Model (IDM)**, a well-established traffic-physics formula that governs how a driver speeds up, slows down, and keeps a following distance based on the car ahead. A bus and a two-wheeler don't drive identically — each vehicle type has its own tuned parameters (desired speed, following distance, aggressiveness, lane discipline) **[VERIFIED — `engine/src/agent/IDM.h:20-45`, `get_default_idm_params()`]**. A "chaos coefficient" per city scales how strictly drivers keep lane discipline — Chicago runs at `chaos: 0.1` (orderly), Ahmedabad at `chaos: 0.4` (much less orderly), reflecting real observed driving-culture differences **[VERIFIED — `cities.yaml:3,28`]**.
2. **It's calibrated against reality, not just internally consistent.** The project doesn't just assert the simulation "looks right" — a validation script runs the engine headlessly and compares its simulated journey times, corridor by corridor, against real trip data from Chicago's Transportation Network Provider (TNP) dataset, and reports the percentage error **[VERIFIED — `pipeline/src/validate.py:85-150`, function `build_report()`]**.

An everyday analogy for the **equity** angle the project cares about: two highways can have the same *average* commute time while one serves mostly wealthy neighborhoods well and leaves poorer neighborhoods stuck — averages hide that. NexusSim computes a **Gini coefficient** (a standard inequality measure borrowed from economics, normally used for income distribution) over per-zone wait times, so "is this policy fair, not just fast" becomes a real, graphable number instead of an afterthought **[VERIFIED — `engine/src/agent/Simulation.h:299-308`, `Simulation::compute_gini()`]**.

1.1 The three subsystems today

Subsystem	Language / stack	Responsibility	Key files
Simulation engine	C++17, CMake	Loads a city's road graph, spawns and moves agents tick-by-tick, streams live state	`engine/src/agent/Simulation.h`, `engine/src/main.cpp`
Data pipeline	Python 3.11	Turns OpenStreetMap + real trip data into the files the engine loads	`pipeline/src/download.py`, `clean.py`, `lanes.py`, `zones.py`, `export.py`, `download_od.py`, `od_matrix.py`, `validate.py`
Dashboard	React 19 + TypeScript + Vite + Leaflet	Renders the live simulation and metrics for a human	`dashboard/src/hooks/useWebSocket.ts`, `dashboard/src/components/Map.tsx`, `MetricsPanel.tsx`, `EquityOverlay.tsx`, `ViewportBoundsSender.tsx`

1.2 What's already validated [VERIFIED]

- Real Chicago OD (origin-destination) trip demand drives agent spawning, at the correct time-of-day rate, via `Simulation::spawn_agents_from_od()` and `Simulation::update_od_spawns()` (`engine/src/agent/Simulation.h:70-124, 392-409`).

- The validation checkpoint is explicit and machine-checked: *MAPE ≤ 25% AND ≥ 75% of corridors within 25% of observed journey time* (`pipeline/src/validate.py:117-150`, the `checkpoint` and `passes_checkpoint` fields).
- 20,000-agent stress testing exists (`engine/tests/test_stress.cpp`, currently untracked in git — see §11) and a quadtree benchmark harness (`engine/bench/bench_quadtree.cpp`) backs the performance claims in §3.2's non-functional requirements.

1.3 Where the four new subjects attach [VERIFIED hooks / PLANNED subsystems]

Subject	Attachment point that exists today	Status
Reinforcement Learning	`SignalController` runs once per intersection every tick (`Simulation.h:132-133`, `signals_` map at `Simulation.h:371`) — a drop-in replacement target	**[PLANNED]** — no learning code exists yet
Soft Computing	`validate.py`'s manual parameter sweep (`main()`, `validate.py:161-236`) is already an optimization problem waiting for a search algorithm	**[PLANNED]**
Computer Vision	No existing hook; new addition	**[PLANNED]**
NLP	No existing hook; new addition	**[PLANNED]**

PART 2

System Architecture Today — Verified Against Code

[VERIFIED — NEW section structure, content sourced from direct code reads, not present in this form in the original PDF]

```
pipeline/ (Python)              engine/ (C++17)                dashboard/ (React+TS)
-----------------               -----------------              ---------------------
download.py → clean.py          Graph.h / GraphLoader.cpp      useWebSocket.ts
  → lanes.py → zones.py           loads graph.json               ws://localhost:9001
  → export.py → graph.json      Simulation.h                     parses {agents,metrics,
download_od.py → od_matrix.py     .tick(dt) every 0.1s             zone_metrics}
  → od_matrix.json              IDM.h car-following            Map.tsx (Leaflet)
validate.py                     Pathfinder.h A* + stochastic     MetricsPanel.tsx (Recharts)
  build_report() vs ground truth Quadtree.h spatial index        EquityOverlay.tsx
                                  SignalController.h Webster's  ViewportBoundsSender.tsx
                                  WebSocketServer.h (uWebSockets)   sends viewport bounds
                                    broadcast_state() → JSON          for LOD culling
```

2.1 Engine tick loop [VERIFIED — `Simulation.h:126-193`, `Simulation::tick()`]

Each 0.1-second tick, in order:

1. Advance sim time; spawn any OD-driven agents due this instant (`update_od_spawns`, line 129).
2. Tick every intersection's `SignalController` (line 133).
3. Snapshot every active agent's position/velocity into thread-safe arrays (lines 135–155) — this snapshot exists specifically so the next step can read positions in parallel without data races.
4. Rebuild the quadtree from the snapshot (line 158).
5. Update all agents in parallel across CPU cores via `std::async` chunks (lines 161–181) — each agent computes its IDM acceleration, checks the leading vehicle via a quadtree radius query, checks whether its next intersection is green, and may attempt a lane change.
6. Recompute zone-level wait-time and Gini metrics every 10th tick (line 192, `compute_zone_metrics()` at line 703).

2.2 Signal control today [VERIFIED — `engine/src/agent/SignalController.h`]

Intersections use **Webster's formula**, a real, decades-old traffic-engineering method for computing fixed-cycle green-light durations from expected traffic volume (`calculate_webster_timing()`, lines 35–73). It is computed **once, at startup**, from approach-edge volumes estimated by clustering incoming-edge angles into two phases (`Simulation::init_signals()`, `Simulation.h:448-490`) — it never adapts to what's actually happening at the intersection afterward. This is precisely the fixed, non-adaptive behavior that the RL work in §5 targets for replacement.

2.3 WebSocket protocol today [VERIFIED — `engine/src/network/WebSocketServer.h`]

- Outbound: `broadcast_state()` (`Simulation.h:195-266`) emits one JSON frame per tick: `{tick, agents:[{id,lat,lon,heading,type,speed}], metrics:{avg_speed,active_agents, completed_agents,avg_wait_time,gini_coefficient}, zone_metrics:[{zone_id,wait_time}]}`.
- Inbound: the server's `message` handler (`WebSocketServer.h:43-63`) currently recognizes exactly one message type, `{"type":"bounds", min_lat, min_lon, max_lat, max_lon}`, used for viewport-based level-of-detail culling — agents outside the dashboard's visible map area are omitted from the next broadcast (`Simulation.h:198-215`).
- The dashboard's `useWebSocket.ts` hook (lines 56–67) reads `data.agents` and `data.metrics` and silently ignores any other top-level JSON keys — this is *why* the PLANNED additions in §5–§8 (`mode`,

`signals`, `incidents`, `cv_congestion`) are described as additive/non-breaking: the hook does not need to change to tolerate new fields, though it does need new `useState` slots to actually *expose* them to components (`IMPLEMENTATION_PLAN.md` Phase 12.4, 17.4).

2.4 Config-driven city pattern [VERIFIED — `cities.yaml`]

Three cities are declared: `chicago` (fully wired — Socrata TNP data source, see `cities.yaml:4-17`), `paris` and `ahmedabad` (stubs with documented data gaps and fallback notes, `cities.yaml:18-35`). This is the established "add a city via config, not code" pattern that every PLANNED addition below is designed to extend (e.g. a future `cv_bbox` key) rather than bypass.

PART 3

Requirements [NEW]

The original PDF described intent narratively but did not enumerate testable requirements. This section derives them from `docs/PRD.md`, `docs/TECH_STACK.md`, the conversation's confirmed scope decisions, and the verified code behavior in §2. Each requirement has an ID for traceability.

3.1 Functional Requirements (FR)

ID	Requirement	Status	Acceptance criteria
FR-1	The engine SHALL simulate heterogeneous agent types (car, bus, auto-rickshaw, two-wheeler, pedestrian) with per-type IDM parameters	**[VERIFIED]**	`IDM.h:20-45` defines 5 distinct parameter sets; `Simulation::spawn_agents` assigns type via `AgentType`
FR-2	The engine SHALL route agents via shortest-path search with an option for stochastic route diversity	**[VERIFIED]**	`Pathfinder::compute_path` (A*, `Pathfinder.h:23`) and `compute_path_stochastic` (`Pathfinder.h:63`) both exist and are used (`Simulation.h:55, 426`)
FR-3	The engine SHALL compute and broadcast citywide and per-zone equity metrics (Gini coefficient) at a bounded interval	**[VERIFIED]**	`compute_zone_metrics()` runs every 10 ticks (`Simulation.h:192`); Gini included in every broadcast (`Simulation.h:253`)
FR-4	The pipeline SHALL calibrate and validate simulated journey times against real-world OD ground truth per corridor	**[VERIFIED]**	`validate.py` checkpoint: MAPE ≤ 25% AND ≥ 75% corridors within 25% (`validate.py:117-150`)
FR-5	The dashboard SHALL render live agent positions and metrics with automatic reconnection on WebSocket drop	**[VERIFIED]**	`useWebSocket.ts:69-86`, exponential backoff capped at `MAX_RECONNECT_DELAY_MS`
FR-6	The system SHALL support adding a new city via configuration only, without code changes	**[VERIFIED]**	`cities.yaml` pattern; `IMPLEMENTATION_PLAN.md` Phase 1.8, 6.8
FR-7	The engine SHALL expose a pluggable signal-control interface so Webster's, fuzzy, and RL strategies can run interchangeably and be switched at runtime without restarting the simulation	**[PLANNED]**	`IMPLEMENTATION_PLAN.md` Phase 12; success = `policy_switch` WS message changes behavior visible in the next broadcast frame
FR-8	An RL policy SHALL control signal timing per intersection, trained against a network-wide shared reward, and SHALL be swappable against the Webster's-formula baseline for direct comparison	**[PLANNED]**	Phase 7, 8, 12
FR-9	A genetic algorithm SHALL replace manual parameter search in the calibration workflow, optimizing `speed_factor`, `route_spread`, `chaos`, `demand_scale` against the existing MAPE checkpoint	**[PLANNED]**	Phase 13; success = GA-tuned MAPE ≤ manually-tuned MAPE from Phase 6
FR-10	A fuzzy-logic controller SHALL provide a third, non-learned signal-control strategy comparable against Webster's and RL	**[PLANNED]**	Phase 13.5–13.6
FR-11	A CV pipeline SHALL classify real-world road congestion from traffic-tile imagery, using both a classical (thresholding) and a learned (CNN) method, for direct comparison	**[PLANNED]**	Phase 14; deliverable = accuracy comparison table
FR-12	A CV pipeline SHALL render a synthetic top-down view of the live simulation and perform real object detection/counting on it	**[PLANNED]**	Phase 15; deliverable = live annotated panel with running count
FR-13	An NLP interface SHALL answer natural-language queries about live simulation state (e.g. "which zone has the worst wait time right now")	**[PLANNED]**	Phase 16; success = correct answers on a held-out query set
FR-14	An NLP interface SHALL parse free-text incident reports into a structured spec and apply a temporary effect to the live simulation's road network	**[PLANNED]**	Phase 17; success = submitted incident visibly changes routing/queueing and expires after its stated duration

ID	Requirement	Status	Acceptance criteria
FR-15	The four subsystems SHALL be demonstrably interoperating (e.g. an NLP incident affects RL agent behavior and dashboard state within the same run), not four isolated demos	**[PLANNED]**	Phase 18; confirmed scope decision — user requested "one cohesive system"
FR-16	The system SHALL implement a documented inventory of 12 RL algorithms total (MAPPO + 11 new) spanning tabular value (Q-Learning, SARSA), deep value (DQN, DDQN, Dueling DQN), policy-gradient (REINFORCE), actor-critic (A2C), PPO-family (single-agent PPO, MAPPO), and continuous-control (SAC, TD3, DDPG) families	**[PLANNED]**	Phases 19–24; acceptance = 9 signal-control algorithms each report an eval vs. Webster baseline; MAPPO/PPO/DQN deploy to the engine via ONNX (Phase 23); SAC/TD3/DDPG train on a standard continuous env (Phase 22)

3.2 Non-Functional Requirements (NFR)

ID	Requirement	Status	Acceptance criteria
NFR-1	The engine SHALL sustain ≥ 20,000 agents without crashing over extended runs	**[VERIFIED — target]**	`engine/tests/test_stress.cpp` (20,000 agents, 500 ticks); target from `docs/PRD.md §7`
NFR-2	Simulation throughput SHALL sustain 60 fps at 20,000 agents on a mid-range dev machine	**[stated target, not independently re-verified this session]**	`docs/PRD.md §7`; measured via `Simulation::avg_tick_ms()/p95_tick_ms()` (`Simulation.h:310-323`)
NFR-3	Quadtree-based proximity queries SHALL outperform naive O(N²) search by ≥ 10× at 10,000 agents	**[stated target]**	`engine/bench/bench_quadtree.cpp`; `docs/PRD.md`, `IMPLEMENTATION_PLAN.md` Phase 3
NFR-4	RL policy inference SHALL complete within a bounded per-tick latency budget so it never blocks the real-time simulation loop	**[PLANNED target: p95 ≤ 8 ms]**	`IMPLEMENTATION_PLAN.md` Phase 8.6; enforced by ONNX Runtime inference running off the hot path (§5)
NFR-5	New WebSocket broadcast fields SHALL be additive and SHALL NOT break existing dashboard consumers	**[VERIFIED as a design constraint]**	`useWebSocket.ts:59-63` already ignores unrecognized keys — verified current behavior that the requirement relies on
NFR-6	External data sources used for Computer Vision SHALL be accessed through documented, Terms-of-Service-compliant APIs	**[PLANNED constraint — see §11]**	No scripted scraping of Google Maps' interactive traffic layer; use Mapbox Traffic Tiles or TomTom Traffic Flow API instead
NFR-7	Simulation validation error SHALL remain within the existing accuracy checkpoint after any new calibration or ML integration	**[VERIFIED checkpoint, PLANNED enforcement]**	MAPE ≤ 25%, `validate.py:117-150`; GA optimizer (FR-9) must not be permitted to regress this

3.3 Technical Requirements (TR)

ID	Requirement	Status
TR-1	Engine: C++17, built with CMake	**[VERIFIED — `engine/CMakeLists.txt`]**
TR-2	Engine WebSocket transport: uWebSockets, JSON payloads via `nlohmann::json`	**[VERIFIED — `WebSocketServer.h:11`]**
TR-3	Pipeline: Python 3.11	**[VERIFIED — `docs/TECH_STACK.md`]**
TR-4	Dashboard: React 19 + TypeScript + Vite + Leaflet + Recharts	**[VERIFIED — package structure under `dashboard/src`]**
TR-5	RL training SHALL occur offline in Python (PyTorch); runtime inference SHALL occur in C++ via ONNX Runtime, never blocking the tick loop with a live Python round-trip	**[PLANNED, justified by NFR-4]**
TR-6	New Python ML/CV/NLP services SHALL run as independent sidecar processes (own ports), not embedded in the dashboard bundle or the C++ engine	**[PLANNED — design decision]**
TR-7	All new per-city configuration SHALL extend `cities.yaml` rather than hardcoding city-specific values	**[PLANNED, consistent with FR-6]**

3.4 Business / Academic Requirements (BR)

ID	Requirement	Status
BR-1	The project SHALL demonstrably cover four academic subjects: Reinforcement Learning, Computer Vision, NLP, Soft Computing	**[confirmed scope]**
BR-2	Depth per subject SHALL be "medium" — a mix of breadth and depth, not a minimal stub nor a full research contribution	**[confirmed scope]**
BR-3	The four subjects SHALL form one integrated system suitable for a single demo narrative, not four independently graded modules	**[confirmed scope]**
BR-4	Work SHALL be structured for parallel execution across a 3–4 person team with a shared foundation phase first	**[confirmed scope; `IMPLEMENTATION_PLAN.md` Phase 12, team roadmap in §10]**
BR-5	RL scope SHALL be independent per-intersection agents trained against a shared network-wide reward — "ambitious but feasible," not full cooperative message-passing MARL and not a single centralized controller	**[confirmed scope]**
BR-6	The system SHALL NOT depend on data sources that violate third-party Terms of Service (specifically: no scripted scraping of Google Maps' live traffic layer)	**[confirmed constraint — see NFR-6]**

PART 4

Vision: One System, Not Four Demos

[VERIFIED unchanged from original PDF intent, expanded with analogy]

A useful mental model: imagine four instruments in an orchestra rehearsing separately in different rooms versus playing the same piece together. Four separate demos prove each instrument *works*; one integrated demo proves the *system* works — which is what BR-3 requires. Concretely, in this project:

- An **NLP incident report** ("accident on Michigan Ave, lane closure") changes the live road network, which the **RL signal policy** reacts to automatically because its observations (queue lengths) shift — no separate wiring needed between NLP and RL beyond both touching the same live simulation state (FR-14, FR-8).
- **CV-derived real-world congestion data** feeds into the **Soft Computing** genetic algorithm as an additional calibration signal (FR-9, FR-11), tying real-world sensing to simulation realism — a link the original engine's `validate.py` process does not have today.
- **RL**, **fuzzy-logic**, and classical **Webster's** signal strategies all implement the same pluggable interface (FR-7) and can be swapped live for a direct side-by-side comparison.
- A **synthetic virtual camera** (FR-12) watches the live simulation itself, carrying zero risk to the control path — it can be demoed in isolation even if other subsystems are behind schedule.

[VERIFIED — flagged data-source correction, unchanged from original] One correction that still applies: pulling live traffic conditions from Google Maps does not qualify as genuine Computer Vision work, and is not accessible in a Terms-of-Service-compliant, scriptable way — the Static Maps API exposes no live-traffic layer, and scripting the interactive JS map view would violate Google's ToS (BR-6, NFR-6). The plan substitutes **Mapbox Traffic Tiles** or the **TomTom Traffic Flow API**, both of which expose equivalent color-coded congestion imagery through documented, compliant APIs.

PART 5

Reinforcement Learning — Adaptive Signal Control [PLANNED]

What it replaces [VERIFIED baseline]: the current fixed-cycle Webster's-formula timing (`SignalController.h:35-73`), computed once at startup and never adaptive afterward (§2.2).

Analogy: Webster's formula is like a traffic light on a timer set once during rush hour and never touched again — it doesn't know if a Tuesday afternoon has unusually light traffic. An RL agent is closer to an experienced human traffic officer who watches the actual queues at the intersection and decides, moment to moment, "let this direction go a little longer" or "switch now" — except here that judgment is learned from thousands of simulated hours rather than years on the job.

Design (BR-5 — independent agents, shared reward):

Element	Design choice	Rationale
Scope	One independent RL agent per signalized intersection	Satisfies "multi-agent" without full inter-agent messaging (BR-5)
Decision timing	At phase boundaries, not every 0.1s tick	Matches `SignalController`'s existing phase/state-machine granularity (`SignalController.h:75-95`)
Action space	`{EXTEND current phase, SWITCH to next phase}`	Small, stable discrete space — avoids an unstable continuous-timing action
Observation	Per-approach queue length (via the existing quadtree, `Quadtree.h`), current phase, time in phase, time of day, neighbor pressure	Reuses infrastructure already in the engine (§2.1 step 4)
Reward	Local avg wait + weighted network-wide avg wait/Gini	Reuses `avg_wait_time_`/`gini_coefficient_`, already computed every 10 ticks (`Simulation.h:192, 374-375`)
Training (TR-5)	Offline in Python, Gym-style env wrapping the engine's WebSocket interface	The 0.1s real-time tick budget cannot tolerate a live Python round-trip
Runtime (TR-5, NFR-4)	Exported to ONNX; C++ inference via `InferenceEngine.h` (new), target p95 ≤ 8 ms	No Python process in the simulation's hot path

Baseline comparison: because Webster's and the RL policy both implement the FR-7 pluggable interface, the dashboard can switch an intersection — or the whole city — between them live and show the wait-time/equity delta immediately (`IMPLEMENTATION_PLAN.md` Phase 12.6).

5.1 From one algorithm to a 12-algorithm inventory [PLANNED — FR-16]

The original scope above specified exactly one RL algorithm — MAPPO. The confirmed scope now extends this to a documented inventory of **12 RL algorithms total** (MAPPO plus 11 new), spanning the major algorithm families so breadth and depth can both be demonstrated (BR-2, FR-16). All nine signal-control algorithms train against the existing Gym-style environment (`ml/env/`) and are compared against the Webster fixed-cycle baseline through one shared harness (`IMPLEMENTATION_PLAN.md` Phases 19–21, 24).

#	Algorithm	Family	Deployment
1	MAPPO (existing)	Multi-agent on-policy	Full path: Python → ONNX → C++ engine
2	PPO (single-agent)	On-policy actor-critic	Full path: Python → ONNX → C++ engine
3	Q-Learning	Tabular value	Python + eval report
4	SARSA	Tabular value	Python + eval report
5	DQN	Deep value	Full path: Python → ONNX → C++ engine
6	DDQN	Deep value	Python + eval report
7	Dueling DQN	Deep value	Python + eval report

#	Algorithm	Family	Deployment
8	REINFORCE	Policy gradient	Python + eval report
9	A2C	Actor-critic	Python + eval report
10	SAC	Continuous off-policy	Showcase (Pendulum-v1, Stable-Baselines3)
11	TD3	Continuous off-policy	Showcase (Pendulum-v1, Stable-Baselines3)
12	DDPG	Continuous off-policy	Showcase (Pendulum-v1, Stable-Baselines3)

Three of them — MAPPO, PPO, DQN — get the full C++ ONNX deployment path (Phase 23) and appear in the dashboard `PolicyComparisonPanel` alongside Webster's and the fuzzy controller. The three continuous algorithms (10–12) are a breadth showcase via Stable-Baselines3 on a standard continuous env (Phase 22), deliberately independent of the signal-control stack.

PART 6

Computer Vision — Two Complementary Pipelines [PLANNED]

6.1 Real-world congestion classification (FR-11)

Traffic-flow tile imagery (color-coded by congestion level, from Mapbox or TomTom per NFR-6) is classified two ways, deliberately built as a comparison:

- **Classical** — convert to HSV color space, threshold road-colored pixels, bucket into a congestion level. No training data required.
- **Learned** — a small CNN or fine-tuned ResNet-18, trained on a hand-labeled sample.

Analogy: this is the difference between a simple rule ("more red pixels on the road = more traffic") and teaching a model to recognize congestion the way a human glancing at a live camera feed would, even under lighting or angle variation the simple rule might miss. Building both and comparing their accuracy is itself the point — it's the classic "hand-crafted features vs. learned features" story that Computer Vision coursework is built around.

Output feeds the Soft Computing GA (§8) as a real-world sanity check on simulated congestion patterns (FR-9).

6.2 Synthetic virtual camera (FR-12)

A second, independent pipeline renders a top-down view directly from the live simulation (roads from `graph.json`, agents as colored shapes) and runs genuine OpenCV detection (contour/blob detection with color segmentation) on that rendered frame — exactly as if watching a real traffic camera, except the "camera" is pointed at the simulation. Because it only reads the same live WebSocket stream the dashboard already reads (§2.3), it carries no risk to the simulation's control path and can be built and demoed independently of everything else.

PART 7

NLP — Two Complementary Interfaces [PLANNED]

7.1 Live metrics chat (FR-13)

Answers questions like "which zone has the worst wait time right now" against the simulation's *actual current state*, not a static snapshot — the service is itself a WebSocket client of the engine, caching the latest `metrics/zone_metrics` broadcast (§2.3). Core: a small, reliable rule-based intent classifier (a fixed set of question types matched by keyword/pattern), with an optional LLM layer on top for a direct "classical NLP vs. LLM" comparison — the system still works with no API key configured, falling back to the rule-based path.

Analogy: think of a restaurant host who knows the fixed set of questions guests always ask ("table for how many," "how long is the wait") cold, versus a concierge who can handle anything — the rule-based path is the host (fast, reliable, limited), the optional LLM path is the concierge (flexible, but a dependency you don't want to be required for the core demo).

7.2 Incident reports (FR-14)

A free-text incident report — "accident on Michigan Ave, lane closure" — is parsed into a structured spec: which road segments are affected (fuzzy-matched against real street names extracted from `graph.json`), what kind of incident, and how long it lasts. That spec is sent to the running simulation, which temporarily reduces speed/capacity on the affected roads. This is the clearest cross-subsystem demo in the project (§4): one text box visibly changes routing on the map, is picked up automatically by the RL policy's observations, and appears in the dashboard's live incident list.

PART 8

Soft Computing — Optimization Without Gradients [PLANNED]

8.1 Genetic algorithm calibration (FR-9)

`validate.py`'s manual parameter sweep (`speed_factor`, `route_spread`, `chaos`, `demand_scale` — `validate.py:161-236`) becomes a genetic algorithm: each candidate parameter set is a chromosome, its fitness is the accuracy score from the *existing* validation checkpoint (`build_report()`, `validate.py:85-150`), and the population evolves via selection, crossover, and mutation — each candidate evaluated as an independent, parallelizable simulation run.

Analogy: rather than a person nudging four dials by hand and re-running the simulation to see if it got closer to reality (the current process), dozens of "attempts" run in parallel each generation, the best-performing attempts "breed" (their parameter values mix), and a few mutate randomly to avoid getting stuck — the same logic evolution uses to search a huge space without gradient information, applied to a search space where there's no clean derivative to follow (the simulation is a black box from the optimizer's point of view).

8.2 Fuzzy-logic signal controller (FR-10)

A second, non-learned alternative to both Webster's formula and the RL policy: reasoning about "long" or "short" queues and wait times using fuzzy membership functions and a small human-readable rule base (e.g. "if the queue is long and the wait is long, extend the green phase significantly"), producing a concrete green-time extension through centroid defuzzification. It slots into the same FR-7 pluggable interface as Webster's and RL.

PART 9

Integrated Architecture and Data Flow

[VERIFIED wiring mechanism / PLANNED new components]

The extension is wired through the engine's **existing WebSocket JSON protocol** (§2.3) and the **existing `cities.yaml` config-driven pattern** (§2.4, TR-7), rather than inventing new plumbing — this reuse is itself load-bearing for NFR-5.

From	To	What flows	Status
NLP incident parser	Engine (WebSocket)	Structured incident spec — mutates live road conditions	**[PLANNED]**
Engine	Dashboard, NLP chat service, CV virtual camera	Live agent positions and metrics — shared real-time state	**[VERIFIED transport, PLANNED consumers]**
CV real-world congestion pipeline	Soft Computing GA, RL reward shaping	Real-world congestion levels — offline calibration/training signal only	**[PLANNED]**
Soft Computing GA	Engine startup parameters	Tuned calibration parameters	**[PLANNED]**
RL training	Engine (ONNX policy file)	A trained signal-control policy the engine loads and runs natively	**[PLANNED]**

Two categories of new components, matching TR-6:

- **Engine-native additions** (real-time, C++): the FR-7 pluggable signal-policy interface, and an "apply incident" hook (FR-14) that temporarily changes road-edge speed/capacity.
- **Python sidecar services** (non-real-time): NLP chat/incident service, CV virtual-camera service — each a lightweight WebSocket client of the engine, exposing its own REST/WS endpoint.
- **Offline/batch pipeline stages**: real-world CV congestion classification, GA calibration — Python-only, feeding static JSON reports.

PART 10

Implementation Timeline [NEW]

Derived from `IMPLEMENTATION_PLAN.md` (now the sole copy, at `docs/IMPLEMENTATION_PLAN.md`). Phases 0–11 are the existing, already-built plan; Phases 12–18 are the four-subject extension added in this planning cycle. Durations are the plan's own estimates; "Depends on" reflects the plan's stated task ordering (e.g. Phase 13's fuzzy controller requires the Phase 12 `SignalPolicy` interface to exist first).

10.1 Already complete (Phases 0–6) — condensed

Phase	Objective	Key deliverable	Duration
0	Repo/CI foundation	`make test` passes on CI	3–4 days
1	Graph loading	Engine prints real city graph stats	1 week
2	Agents + pathfinding	500 agents navigate, no crashes	1.5 weeks
3	Quadtree + scale	Quadtree ≥ 10× faster at 10k agents	1 week
4	WebSocket + dashboard map	5k agents on real map at 60 fps in browser	1.5 weeks
5	Fixed-cycle signal baseline	Efficiency + equity panels live	1 week
6	Real OD demand + validation	Chicago validation report within 25% error	1 week

10.2 In the existing plan but not yet built (Phases 7–11)

Phase	Objective	Key deliverable	Duration
7	MARL training environment	Rising reward curve on toy graph (MLflow)	1.5 weeks
8	ONNX export + C++ inference	AI mode running, p95 inference ≤ 8 ms	1 week
9	Multi-city training	MARL vs. Webster's table across 3 cities	1.5 weeks
10	Policy toggles + polish	Toggle → metrics change → export PDF	1 week
11	Hardening + docs + demo	`make demo` works from clean clone < 30 min	1 week

10.3 Four-subject extension (Phases 12–18) — full detail

Phase	Objective	Key deliverables	Depends on	Duration	Success criteria	
12 — Signal Policy Abstraction	Make signal control pluggable so Webster's/Fuzzy/RL are hot-swappable	`SignalPolicy` interface; `WebsterPolicy` (behavior-preserving extraction); `broadcast_state()` gains `mode`/`signals`; `policy_switch` WS message; `PolicyComparisonPanel.tsx`	Phase 5 (existing signal baseline)	3–4 days	Engine with `--signal-policy webster` is byte-identical to pre-refactor behavior; `policy_switch` visibly changes behavior with no restart	
13 — Soft Computing	Replace manual calibration with a GA; add a fuzzy signal strategy	`optimize_calibration.py`; `ga_calibration_report.json` with convergence curve; `FuzzyPolicy.h`; `CalibrationReportPanel.tsx`	Phase 6 (`validate.py`); Phase 12 (for `FuzzyPolicy`)	1.5 weeks	GA-tuned MAPE ≤ manually-tuned MAPE from Phase 6; fuzzy controller runs live via `--signal-policy fuzzy`	
14 — CV: Real-World Congestion	Classify congestion from real traffic-tile imagery	`cv_congestion.json`; classical-vs-CNN accuracy table; `CongestionCVOverlay.tsx`	Phase 13 (feeds GA fitness)	1 week	Classification pipeline runs end-to-end per city; accuracy comparison documented	
15 — CV: Synthetic Virtual Camera	Live detection/counting on a rendered top-down sim view	`virtual_camera.py`; `virtual_camera_service.py`; `VirtualCameraPanel.tsx`	Phase 4 (WebSocket stream to consume)	1 week	Detected count tracks actual simulated traffic in view, logged error %	
16 — NLP: Live Metrics Chat	Answer NL questions about live sim state	`chat_service.py`; rule-based intent classifier; optional LLM layer; `ChatPanel.tsx`	Phase 5/6 (metrics to query)	1 week	Correct answers on a held-out query test set	
17 — NLP: Incident Reports	Free text mutates the live simulation	`incident_parser.py`; `apply_incident()` in engine; `incidents[]` in broadcast; `IncidentReportPanel.tsx`	Phase 12 (control-message pattern), Phase 16 (shares the NLP service)	1 week	Submitted incident visibly changes routing; expires after stated duration	
18 — Cross-Subsystem Integration	Prove all four subjects interoperate in one run	Extended `make demo`; combined results writeup; integrated demo recording	Phases 12–17 (all subsystems working individually)	1 week	One continuous recording showing incident → RL reaction → metrics update, plus CV/GA/chat all live together	

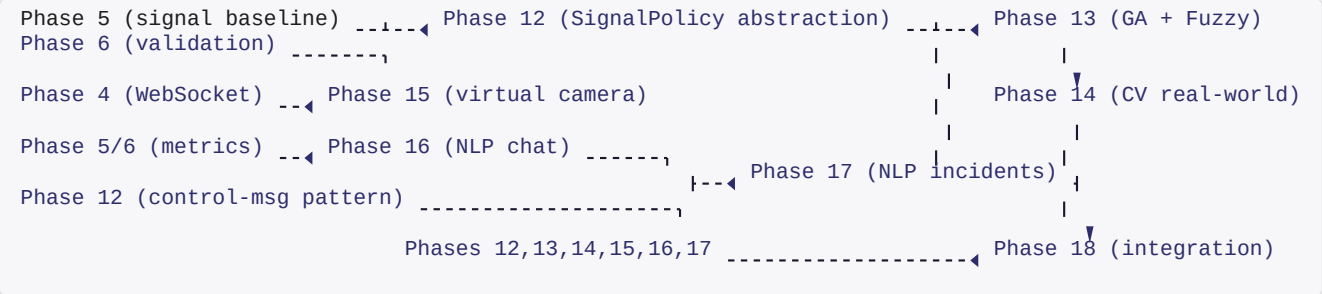
10.4 RL algorithm breadth extension (Phases 19–24) — full detail

The 12-algorithm RL inventory (FR-16) is built as Phases 19–24 on top of the existing RL track. Phases 19–22 are Python-only and unblocked once Phase 7's environment exists (it does). Phase 23 requires Phase 8 (`InferenceEngine`) and Phase 12 (`SignalPolicy`), which are PLANNED.

Phase	Objective	Key deliverables	Depends on	Duration	Success criteria
19 — Shared RL framework	One harness trains/evaluates every algorithm	`BaseTrainer` interface; `ml/algo/configs.yaml`; single-agent env wrapper; `ml/eval/report.py`; `benchmark.py`	Phase 7 (env exists)	3–4 days	`benchmark.py --algos ppo,dqn,ql` prints a comparison table + MLflow

Phase	Objective	Key deliverables	Depends on	Duration	Success criteria
20 — Value-based RL	Tabular + deep value algorithms on signal control	Q-Learning, SARSA, DQN, DDQN, Dueling DQN + unit tests	Phase 19	1 week	5 algorithms trained on toy graph, eval reports vs. Webster
21 — Policy-based RL	Policy-gradient + actor-critic algorithms	REINFORCE, A2C, single-agent PPO	Phase 19	4–5 days	8 signal-control algorithms in the comparison table
22 — Continuous showcase	Breadth via Stable-Baselines3 on a standard env	SAC, TD3, DDPG on Pendulum-v1 + return curves	none	1–2 days	All three learn; returns logged to MLflow
23 — C++ deployment	Headline algorithms run in the engine	Generalized `export_onnx.py`; `InferenceEngine --algo` dispatch; `RLPolicy`	Phase 8, Phase 12	1 week	`--signal-policy rl --algo dqn` runs; p95 ≤ 8 ms (NFR-4)
24 — Results & docs	12-algorithm inventory documented and compared	`docs/results.md` table; ablation write-up; docs updated	Phases 19–23	2 days	12-algo table + ablation; TRD/PRD/US/explained consistent

10.5 Phase-dependency diagram



PART 11

Assumptions and Open Questions [NEW]

This revision was produced without a binary file upload in this conversation. The following assumptions were made, and the listed information would let a future revision remove them:

1. **[ASSUMPTION]** "The attached PDF" refers to `docs/NexusSim_Explained.pdf`, generated earlier in this same project thread via a `reportlab` script (superseded by `docs/NexusSim_Explained_Engineered_v2.pdf`). If a different, externally authored PDF was intended (e.g. one the user has outside this repo), please share it — this revision cannot account for content it has never seen.
2. **[ASSUMPTION]** "The attached implementation plan" refers to `docs/IMPLEMENTATION_PLAN.md` (the single repository copy after the docs consolidation). If a separate implementation-plan document exists elsewhere, its phases/durations may differ from what's reflected in §10.
3. **[ASSUMPTION]** NFR-2 and NFR-3's performance numbers (60 fps at 20k agents, ≥10× quadtree speedup) are carried over from `docs/PRD.md` and `IMPLEMENTATION_PLAN.md` as *stated targets* — they were not re-benchmarked during this session. `engine/bench/bench_quadtree.cpp` and `engine/tests/test_stress.cpp` exist to verify them, but this revision did not execute them.
4. **Untracked files** — `git status` at the time of this revision shows `dashboard/src/components/ViewportBoundsSender.tsx` and `engine/tests/test_stress.cpp` as untracked, and several core files (`Simulation.h`, `WebSocketServer.h`, `main.cpp`, `cities.yaml`, `useWebSocket.ts`, `validate.py`, `run.bat`, `CMakeLists.txt`, `bench_quadtree.cpp`, `Map.tsx`, `MetricsPanel.tsx`) as modified but uncommitted. This revision reflects the **working-tree state on disk**, not the last commit — if the intent was to document the last *committed* state instead, the citations in §1–§2 would need to be re-verified against `git show HEAD:<path>` rather than the working tree.
5. **[ASSUMPTION]** BR-1 through BR-6 are derived from this conversation's confirmed scope decisions (medium depth, one cohesive system, 3–4 person team, independent-agent RL with shared reward, no Google Maps scraping) rather than from a formal requirements document. If a course syllabus or rubric exists with different or additional mandatory requirements, it should be supplied so §3.4 can be corrected against it rather than inferred from conversation.
6. **What would most improve the next revision:** (a) the actual external PDF/plan file if one exists outside this repo, (b) a course rubric or syllabus for BR-1–BR-6, (c) confirmation of whether the untracked/uncommitted files in point 4 should be treated as part of the documented baseline or excluded as work-in-progress.